

CO2-ASSESSMENT TOOL-2

Name: SIVASUDHAN N

Reg.no: 192421017

Course name: DESIGN AND ANALYSIS OF ALGORITHMS

Code: CSA0613

Faculty: Dr. F. MARY HARIN FERNANDEZ

Date: 29-07-2026

Question

Q34. A geographic mapping system identifies the outer boundary of terrain points using a brute-force convex hull algorithm. The number of data points grows continuously.

Answer

(a) Explain how the brute-force convex hull algorithm operates

The Brute-Force Convex Hull Algorithm is used to find the outer boundary (convex hull) of a set of points. It checks every possible pair of points to determine whether the line segment joining them forms an edge of the convex hull.

Working Steps

- 1. Consider every pair of points in the dataset.**
- 2. Draw a line joining the selected pair of points.**
- 3. Check the position of all remaining points with respect to this line.**
- 4. If all other points lie on the same side of the line (or on the line itself), then the selected pair forms an edge of the convex hull.**
- 5. Repeat this process for every pair of points.**
- 6. Combine all valid edges to obtain the convex hull.**

Example

Terrain Points:

A(1,1), B(2,5), C(5,6), D(7,3), E(6,1), F(3,2)

Process:

- Check edge A–B.**

- Verify whether all other points lie on one side of line AB.
- If true, AB is part of the convex hull.
- Repeat for all other point pairs.
- The outer boundary formed is the convex hull.

Time Complexity

- Best Case: $O(n^3)$
- Average Case: $O(n^3)$
- Worst Case: $O(n^3)$

Source Code Screenshot

```

1- def orientation(p, q, r):
2-     return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0])
3-
4- def brute_force_convex_hull(points):
5-     n = len(points)
6-     hull_edges = []
7-
8-     for i in range(n):
9-         for j in range(i + 1, n):
10-             left = right = 0
11-
12-             for k in range(n):
13-                 if k == i or k == j:
14-                     continue
15-
16-                 val = orientation(points[i], points[j], points[k])
17-
18-                 if val > 0:
19-                     left += 1
20-                 elif val < 0:
21-                     right += 1
22-
23-             if left == 0 or right == 0:
24-                 hull_edges.append((points[i], points[j]))

```

OUTPUT

```

Convex Hull Edges:
((1, 1), (2, 5))
((1, 1), (6, 1))
((2, 5), (5, 6))
((5, 6), (7, 3))
((7, 3), (6, 1))

```

(b) Analyze its computational complexity

The brute-force convex hull algorithm performs a large number of comparisons because every pair of points is checked against all remaining points.

Complexity Analysis

- There are approximately $n(n-1)/2$ pairs of points.
- For each pair, all n points are tested.
- Therefore, the overall time complexity becomes:

Time Complexity = $O(n^3)$

Performance Analysis

For Small Datasets

- Produces correct convex hull.
- Easy to understand and implement.
- Suitable for educational purposes.

For Large Datasets

- Number of comparisons increases rapidly.
- Execution time becomes very high.
- Performance degrades significantly as the number of terrain points increases.

Comparison Table

Number of Points Performance

Small Dataset Fast and acceptable

Medium Dataset Moderate execution time

Large Dataset Very slow due to $O(n^3)$ complexity

Program Output Screenshot

```
1 import time
2
3 def orientation(p, q, r):
4     return (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
5
6 def brute_force(points):
7     n = len(points)
8
9     for i in range(n):
10        for j in range(i+1, n):
11            left = right = 0
12
13            for k in range(n):
14                if k == i or k == j:
15                    continue
16
17                val = orientation(points[i], points[j], points[k])
18
19                if val > 0:
20                    left += 1
21                elif val < 0:
22                    right += 1
23
24
```

OUTPUT

```
Number of Points: 10
Time Taken: 0.00015974044799804688 seconds
Time Complexity: O(n^3)
```

(c) Discuss why the approach is inefficient for large datasets

The brute-force convex hull algorithm becomes inefficient as the number of terrain points increases because every pair of points must be compared with all other points.

Reasons

1. It checks every possible pair of points.
2. It performs a large number of unnecessary comparisons.
3. Execution time increases rapidly with larger datasets.
4. High computational cost makes it unsuitable for real-time geographic mapping.
5. More efficient algorithms such as Graham Scan and Jarvis March provide better performance for large datasets.

Analysis/Documentation Screenshot

```
1 sizes = [10, 50, 100, 500, 1000]
2
3 print("Dataset Size\tEstimated Operations (n^3)")
4
5 for n in sizes:
6     operations = n ** 3
7     print(f"{n}\t\t{operations}")
```

OUTPUT

Dataset Size	Estimated Operations (n^3)
10	1000
50	125000
100	1000000
500	125000000
1000	1000000000

Conclusion

The Brute-Force Convex Hull Algorithm is simple and easy to implement for finding the outer boundary of terrain points. However, its $O(n^3)$ time complexity makes it inefficient for large datasets because the number of comparisons grows rapidly. Therefore, while it is useful for small datasets and learning purposes, more efficient convex hull algorithms are preferred for large-scale geographic mapping systems.